A PROJECT REPORT ON

CareerBridge: An Academia–Industry Internship and Placement Portal with Rule-Based Skill Matching

Software Requirements Specification

Smart India Hackathon Problem Statement SIH26044 · Ministry of Ayush

Course Code
INT 253

Course Name
Web Development in Python Using Django

Submitted by
Gourav Dash
Registration No.: 12307549

Prepared for Continuous Assessment 2



TABLE OF CONTENTS

Revision History 2
1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions, Acronyms, and Abbreviations 3
1.4 References 3
1.5 Overview 4
2. General Description 4
2.1 Product Perspective 4
2.2 Product Functions 5
2.3 User Characteristics 5
2.4 General Constraints 5
2.5 Assumptions and Dependencies 5
3. Specific Requirements 5
3.1 External Interface Requirements 5
3.1.1 User Interfaces 5
3.1.2 Hardware Interfaces 6
3.1.3 Software Interfaces 6
3.1.4 Communications Interfaces 7
3.2 Functional Requirements 7
3.2.1 FR-1 User Registration and Authentication 7
3.2.2 FR-2 Student Profile Management 8
3.2.3 FR-3 Skill Management 8
3.2.4 FR-4 Internship Posting 8
3.2.5 FR-5 Internship Search and Discovery 8
3.2.6 FR-6 Skill-Match Scoring 8
3.2.7 FR-7 Application Submission and Tracking 9
3.2.8 FR-8 Applicant Review and Status Management 9
3.2.9 FR-9 Role-Based Dashboards and Analytics 9
3.3 Non-Functional Requirements 9
3.3.1 Performance 9
3.3.2 Reliability 9
3.3.3 Availability 9
3.3.4 Security 9
3.3.5 Maintainability 10
3.3.6 Portability 10
3.4 Design Constraints 10
3.5 Other Requirements 10
4. Analysis Models 10
4.1 Data Flow Diagrams (DFD) 10
4.2 Entity-Relationship Model 11
4.3 System Workflows 12
4.4 Skill-Matching Algorithm Model 15
5. GitHub Link 15
A. Appendices 16
A.1 Appendix 1: Data Dictionary 16
A.2 Appendix 2: Code Review Observations and Future Work 16
References 17

1. INTRODUCTION

Internship and placement processes in Indian higher education are still largely managed through spreadsheets, e-mail threads and notice boards. Students cannot easily see which opportunities match what they already know, recruiters spend time screening applicants who lack the core skills for a role, and universities have no consolidated view of placement outcomes. The Smart India Hackathon problem statement SIH26044, issued by the Ministry of Ayush, asks for an academia–industry collaboration portal that closes this gap.

CareerBridge is the author's response: a web platform on which students, companies and universities share one data model. This document is the Software Requirements Specification (SRS) for that platform. It follows the structure of IEEE Std 830 [1] and was prepared by reading the complete source repository, so every requirement below is traceable to code that exists today, and gaps are reported in Appendix A.2 rather than described as finished features.

1.1 Purpose

The purpose of this SRS is to define, in a precise and verifiable way, what CareerBridge must do and the conditions under which it must operate. It records the functional behaviour of each module, the interfaces between the Next.js frontend and the Django backend, the data the system stores, and the quality attributes the system is expected to meet.

The intended audience is: (1) the developer, as the reference for design, implementation and testing; (2) the course faculty and evaluators of Continuous Assessment 3; (3) the client or problem-statement owner, to confirm that the product meets their needs; and (4) future maintainers who will extend the platform.

1.2 Scope

The software product is named **CareerBridge — Academia–Industry Internship & Placement Portal**. It consists of a REST API server (Django 6 with Django REST Framework) and a browser client (Next.js 16).

What the product does. CareerBridge lets students register, complete an academic profile, record skills with a proficiency level, search and filter published internships, view a skill-match score for any internship, apply with a cover letter and track every application. It lets company recruiters post internships as drafts or published listings, review applicants and update each application's status with private notes. It gives university representatives platform-wide placement statistics. Each role receives its own dashboard of key performance indicators (KPIs).

What the product does not do in the current version: it does not process payments, schedule interviews, send e-mail or SMS notifications, parse résumés automatically, or use machine learning for matching. Companies, universities, skills and company memberships are master data that are not yet editable from the web interface (see Appendix A.2).

Benefits and goals. The measurable objectives are: a student can go from registration to a submitted application in under five minutes; a recruiter can see every applicant for a listing on one page and change a status in two clicks; the match score is fully explainable, listing the exact required skills a student is missing; and a university can read its placement rate and top ten student skills from a single dashboard.

1.3 Definitions, Acronyms, and Abbreviations

TABLE I
DEFINITIONS AND ACRONYMS

Term	Meaning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DRF	Django REST Framework
ER	Entity-Relationship (model)
JWT	JSON Web Token, a signed token used for stateless authentication [9]
KPI	Key Performance Indicator
ORM	Object-Relational Mapper
Access token	Short-lived JWT (1 hour) sent with each API call
Refresh token	Longer-lived JWT (7 days) used to obtain a new access token
Recruiter	User with role company who belongs to a company
Required skill	Skill an internship needs; weight 3 in matching
Preferred skill	Skill that is useful but optional; weight 1
Match score	0–100 value of how well a student's skills fit an internship
Placement rate	Selected applications ÷ all applications × 100
SIH	Smart India Hackathon
SRS	Software Requirements Specification

1.4 References

All documents referred to in this SRS are listed in the References section at the end, numbered [1]–[12]. They comprise the IEEE recommended practice for

SRS documents [1], the official documentation of the frameworks and libraries used (Django [2], Django REST Framework [3], SimpleJWT [4], Next.js [5], React [6], shadcn/ui [7], Recharts [8]), the JWT and HTTP specifications [9], [10], the OWASP Top 10 [11] and the project repository [12]. Online documents can be obtained from the URLs given; the IEEE standard is available from the IEEE Xplore digital library.

1.5 Overview

Section 2 gives a general description of the product: its context, main functions, users, constraints and assumptions. Section 3 contains the specific requirements: external interfaces, nine functional requirements written as introduction, inputs, processing, outputs and error handling, non-functional requirements, design constraints and other requirements. Section 4 presents the analysis models: data flow diagrams, the entity-relationship model, the end-to-end workflows of each role and the skill-matching algorithm. Section 5 gives the GitHub link and instructions for running the project. Appendix A contains the data dictionary and the code-review observations; the appendices are informative and are not part of the binding requirements.

2. GENERAL DESCRIPTION

This section describes the general factors that affect the product and its requirements. It does not state specific requirements; it makes those in Section 3 easier to understand.

2.1 Product Perspective

CareerBridge is a new, self-contained product. It is not a component of a larger system, although it is designed to be integrated later with university information systems. It follows a three-tier client–server architecture (Fig. 1). The **client tier** is a Next.js single-page application that renders the landing page, authentication pages, internship listings and the role dashboards. The **application tier** is a Django project, `CareerBridge`, exposing a JSON REST API under `/api/`; it is divided into five Django applications: `accounts`, `institutions`, `internships`, `applications` and `analytics`. The **data tier** is a relational database (SQLite in development, PostgreSQL in production) and a media folder for uploaded photos, logos and résumés.

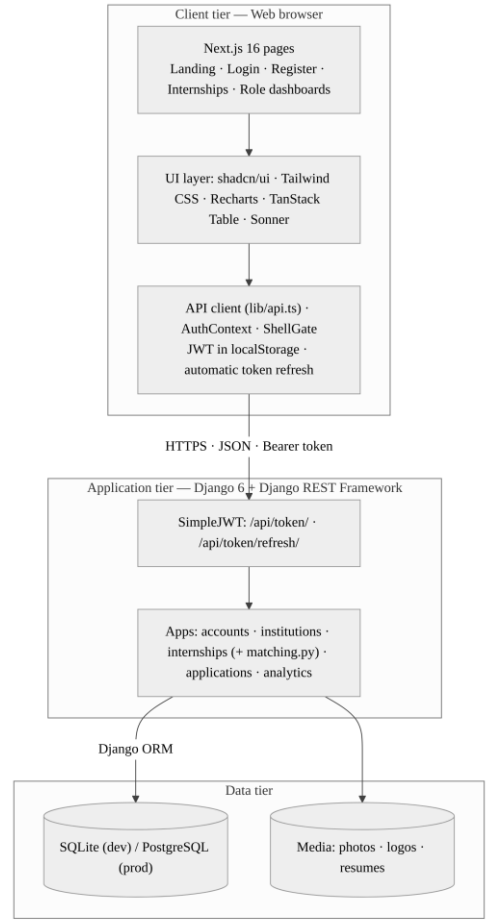


Fig. 1. Three-tier architecture of CareerBridge.

The two tiers communicate only over HTTP with JSON bodies. Every protected request carries a JWT in the `Authorization: Bearer` header, so the server holds no session state for API users.

2.2 Product Functions

The major functions of CareerBridge are summarised below; each is specified in detail in Section 3.2.

- **Account management:** role-based registration (student, company recruiter, university representative), login, token refresh, logout and profile editing.
- **Student profile:** university, degree, branch, graduation year, CGPA, résumé, LinkedIn and GitHub links, bio, availability, and an automatic profile-completion percentage.
- **Skill management:** a shared skill catalogue with search, and per-student skills at beginner, intermediate or advanced level.
- **Internship management:** recruiters create listings with title, description, location, work mode, stipend, duration, openings, deadline and status (draft, published, closed).
- **Discovery:** public, paginated search of published internships by keyword, location, work mode and skill; bookmarking of saved jobs.
- **Skill matching:** a weighted, rule-based score with matched, missing and preferred skills.
- **Applications:** apply with a cover letter, prevent duplicates, track status, withdraw while still pending.
- **Applicant review:** recruiters list and filter applicants and set status and private notes.
- **Analytics:** KPI dashboards for students, recruiters, universities and administrators.

2.3 User Characteristics

CareerBridge has four user classes (Table II). None of them is expected to have technical training; the interface therefore relies on familiar web patterns, clear status labels and helpful empty states.

TABLE II
USER CLASSES

User class	Characteristics and needs
Student	Undergraduate or postgraduate; uses the portal frequently, often on a phone; needs fast search, an honest picture of skill gaps and clear status feedback.
Company recruiter	HR or hiring staff; moderate frequency; needs to post quickly and triage many applicants.
University representative	Training and placement officer; occasional use; needs aggregate statistics rather than individual records.
Administrator	Platform operator; maintains master data (skills, companies, universities, memberships) and verifies companies.

3. SPECIFIC REQUIREMENTS

This section contains the detailed requirements that guide design, implementation and testing. Each functional requirement is uniquely numbered (FR-1 to FR-9) so it can be traced to the API endpoints in Table V and to the test cases.

2.4 General Constraints

- **Regulatory:** personal data of students (contact details, CGPA, résumé) must be handled in line with India's Digital Personal Data Protection Act, 2023.
- **Technology:** the backend must use Python with Django 6.1 or later and Django REST Framework; the frontend must use Next.js 16 with React 19 and TypeScript, as fixed by the project's dependency files.
- **Interface:** all client–server communication is through the documented REST API; the frontend never accesses the database directly.
- **Hardware:** the system must run on a single commodity server for the pilot; no specialised hardware is used.
- **Schedule:** the product is a hackathon minimum viable product (MVP); rule-based matching is chosen over machine learning so that it is transparent and testable within the time available.

2.5 Assumptions and Dependencies

- Users have a modern web browser with JavaScript enabled and local storage available (tokens and saved jobs are kept there).
- Python 3.12 or later and Node.js 20 or later are available on the build and host machines.
- A PostgreSQL server is available in production; if not, SQLite is used, which limits concurrent writes.
- Skills, companies, universities and recruiter–company memberships are created by an administrator before recruiters can post; if this master data is missing, posting and matching cannot work.
- Third-party packages listed in `requirements.txt` and `package.json` remain available from PyPI and npm.

3.1 External Interface Requirements

3.1.1 User Interfaces

The user interface is a responsive web application with a collapsible sidebar, a page header with breadcrumbs, light and dark themes, toast notifications for success and error messages, skeleton loaders while

data is fetched, and a command menu opened with Ctrl+K (⌘K on macOS) for keyboard navigation. The pages and the role that can open each one are listed in Table III.

When the API returns no data (for example on a fresh installation), the dashboards and listings show clearly labelled sample data so the layout can be reviewed; this behaviour is controlled by the environment variable `NEXT_PUBLIC_DEMO_DATA` and can be switched off.

TABLE III
FRONTEND PAGES AND ROUTES

Route	Page	Access	Main content
/	Landing page	Public	Hero, application-pipeline preview, features for each stakeholder, call to action
/login, /register	Authentication	Public	Login form; registration with role selector (student, company recruiter, university representative)
/internships	Internship listings	Public	Search, filters, internship cards, saved jobs, quick apply
/internships/[id]	Internship detail	Public (apply: student)	Description, company, skills, match gauge, how-it-works steps, apply dialog
/dashboard	Role redirect	Logged in	Sends the user to the dashboard of their role
/dashboard/student	Student overview	Student	KPI tiles, status breakdown, recent applications, profile completion
/dashboard/student/profile, /skills	Profile and skills	Student	Academic details form; skill search, add with proficiency, remove
/dashboard/student/applications	My applications	Student	Table of applications with status badges and withdraw action
/dashboard/company	Company overview	Recruiter	Listings, applicant totals, outcomes chart, recent applicants
/dashboard/company/internships, /create	Listings and post form	Recruiter	Own listings table; form to post a new internship
/dashboard/company/applicants	Applicants	Recruiter	Status tabs, applicant table, review sheet with status and notes
/dashboard/university	University overview	University	Students, internships, applications, placements, placement rate, top skills
/dashboard/account	Account	Logged in	User account details

3.1.2 Hardware Interfaces

CareerBridge has no direct hardware interfaces. On the client side it requires only a device able to run a current browser (desktop, laptop, tablet or phone, minimum viewport width 360 px). On the server side a single host with at least 2 vCPU, 2 GB RAM and 10 GB of disk is sufficient for the pilot; uploaded files are stored on the server's file system under `media/`.

3.1.3 Software Interfaces

The software components and their versions are listed in Table IV, and the REST endpoints offered by the backend are listed in Table V. All list endpoints are paginated with 20 items per page (`PageNumberPagination`), and the default permission is *IsAuthenticated* unless an endpoint states otherwise.

TABLE IV
SOFTWARE COMPONENTS

Component	Technology and version
Backend framework	Django ≥ 6.1, Django REST Framework ≥ 3.18
Authentication	djangorestframework-simplejwt ≥ 5.3
Cross-origin access	django-cors-headers ≥ 4.3
Configuration	python-decouple ≥ 3.8 (environment variables)
Database	SQLite (dev); PostgreSQL via psycopg2-binary ≥ 2.9 (prod)
Files and serving	Pillow ≥ 10, Gunicorn ≥ 21.2, WhiteNoise ≥ 6.6
Frontend	Next.js 16.4, React 19.3, TypeScript 5
UI and styling	shadcn/ui, Base UI, Radix, Tailwind CSS 4, lucide and Solar icons
Tables and charts	TanStack Table, Recharts 3, dnd-kit
Feedback and validation	Sonner toasts, Zod schemas, next-themes

TABLE V
REST API ENDPOINTS

Method	Endpoint	Access	Purpose	FR
POST	/api/token/	Public	Log in; returns access and refresh tokens	FR-1
POST	/api/token/refresh/	Public	Exchange refresh token for a new access token (rotated)	FR-1
POST	/api/accounts/register/	Public	Create a user with a role	FR-1
GET	/api/accounts/me/	Authenticated	Current user and role	FR-1
GET, PUT, PATCH	/api/accounts/profile/	Authenticated	Read or update name, e-mail, phone, photo	FR-1
GET	/api/institutions/universities/	Public	List universities	FR-2
GET, PATCH	/api/institutions/student-profile/	Authenticated	Read or update own student profile (created on first read)	FR-2
GET	/api/institutions/companies/	Public	List verified companies	FR-4
GET	/api/internships/skills/?search=	Public	Skill catalogue with search	FR-3
GET, POST	/api/internships/student-skills/	Authenticated	List or add own skills	FR-3
DELETE	/api/internships/student-skills/{id}/	Authenticated	Remove an own skill	FR-3
GET, POST	/api/internships/my/	Recruiter	List or create own company's internships	FR-4
GET	/api/internships/?q=&location=&work_mode=&skill=	Public	Search published internships	FR-5
GET	/api/internships/{id}/	Public	Published internship detail with skills	FR-5
GET	/api/internships/{id}/match/	Authenticated	Skill-match score for the current student	FR-6
GET, POST	/api/applications/	Authenticated	List own applications or apply	FR-7
DELETE	/api/applications/{id}/withdraw/	Authenticated	Withdraw an application still in Applied	FR-7
GET	/api/applications/recruiter/?internship=	Authenticated (scoped)	Applicants for the recruiter's companies	FR-8
PATCH	/api/applications/recruiter/{id}/status/	Authenticated (scoped)	Update status and recruiter notes	FR-8
GET	/api/analytics/student/ | company/ | university/ | admin/	Matching role	Dashboard KPIs for each role	FR-9

3.1.4 Communications Interfaces

All communication uses HTTP/1.1 with JSON request and response bodies (`Content-Type: application/json`) [10]. In production the API and frontend must be served over HTTPS (TLS 1.2 or later). The frontend reads the API base URL from `NEXT_PUBLIC_API_URL` (default `http://localhost:8000/api`). Cross-origin requests are allowed only from the origins listed in `CORS_ALLOWED_ORIGINS` (default `http://localhost:3000` and `http://127.0.0.1:3000`), with credentials enabled. Authentication uses the `Authorization: Bearer <access token>` header. No e-mail, SMS or push channel is used in this version.

3.2 Functional Requirements

The functional requirements are grouped by feature. For each feature the introduction, inputs, processing, outputs and error handling are stated. The workflows that connect these features are drawn in Section 4.3.

3.2.1 FR-1 User Registration and Authentication

3.2.1.1 Introduction. The system shall let a new user create an account with one role and then log in to receive JWT tokens that authorise later requests.

3.2.1.2 Inputs. Registration: username, e-mail, first name, last name, role (student, company or university), optional phone, password and confirmation. Login: username and password.

3.2.1.3 Processing. The server checks that the password has at least 8 characters and that both password fields match, then creates the user with a hashed password. On login, SimpleJWT validates the credentials and issues an access token valid for 1 hour and a refresh token valid for 7 days. The client stores both tokens in local storage, calls `/api/accounts/me/` to load the user and redirects to `/dashboard/{role}`. When any call returns 401, the client requests a new access token once using the refresh token (refresh tokens rotate on every use) and repeats the call (Fig. 5).

3.2.1.4 Outputs. HTTP 201 with the created user on registration; token pair on login; the user object with role from `/me/`.

3.2.1.5 Error Handling. Mismatched or short passwords and duplicate usernames return HTTP 400 with a field message shown in the form. Wrong credentials return 401. If the refresh fails, the client clears the tokens and the dashboard guard sends the user to `/login`.

3.2.2 FR-2 Student Profile Management

3.2.2.1 Introduction. The system shall let a student maintain an academic profile that recruiters can see when reviewing applications.

3.2.2.2 Inputs. University (selected from the list), degree, branch, graduation year, CGPA (up to two decimals), résumé URL or file, LinkedIn URL, GitHub URL, bio and an availability flag.

3.2.2.3 Processing. On first access the server creates an empty profile linked one-to-one to the user. Updates are partial (PATCH). The server computes the profile-completion percentage from eight fields: university, degree, branch, graduation year, CGPA, résumé, LinkedIn and bio.

3.2.2.4 Outputs. The saved profile, including the university object and the completion percentage.

3.2.2.5 Error Handling. Invalid URLs, an unknown university or a CGPA in the wrong format return HTTP 400 with the field in error; the form shows a toast.

3.2.3 FR-3 Skill Management

3.2.3.1 Introduction. The system shall let a student declare skills from a shared catalogue so that matching and analytics can use them.

3.2.3.2 Inputs. A search term for the catalogue; a skill identifier and a proficiency level (beginner, intermediate, advanced); the identifier of a skill to remove.

3.2.3.3 Processing. The catalogue is searched by name and category. Adding a skill creates a StudentSkill row; a database constraint allows each skill only once per student. Removing deletes the row, and only the owner's skills can be deleted.

3.2.3.4 Outputs. The student's skill list with names, categories and levels.

3.2.3.5 Error Handling. Adding a skill twice is rejected by the unique constraint; an unknown skill identifier returns 400; deleting another student's skill returns 404.

3.2.4 FR-4 Internship Posting

3.2.4.1 Introduction. The system shall let a recruiter who belongs to a company post internships for that company.

3.2.4.2 Inputs. Title, description, location, work mode (on-site, hybrid, remote), stipend, duration in months, number of openings, application deadline and status (draft, published, closed).

3.2.4.3 Processing. Only users with role company may use the endpoint. The server finds the user's active company membership and saves the internship with that company and the recruiter as `posted_by`. Drafts are stored but not shown publicly; published listings appear in search immediately.

3.2.4.4 Outputs. The new listing; the recruiter's list of listings for all their companies.

3.2.4.5 Error Handling. A non-recruiter receives 403. A recruiter with no active membership receives 403 "No active company membership found". Missing required fields return 400.

3.2.5 FR-5 Internship Search and Discovery

3.2.5.1 Introduction. The system shall let anyone, including visitors who are not logged in, browse published internships.

3.2.5.2 Inputs. Optional query parameters: keyword `q` (title or description), `location`, `work_mode` and `skill`; an internship identifier for the detail page.

3.2.5.3 Processing. Only internships with status published are returned, newest first, with the company preloaded. Filters are combined with AND; the skill filter removes duplicates. Students can bookmark listings, which are kept in the browser's local storage.

3.2.5.4 Outputs. A paginated list with title, company, logo, location, mode, stipend, duration, deadline, openings and number of required skills; a detail view with company information and required and preferred skills.

3.2.5.5 Error Handling. A draft, closed or non-existent internship returns 404 on the detail endpoint; an empty result shows an empty state.

3.2.6 FR-6 Skill-Match Scoring

3.2.6.1 Introduction. The system shall tell a student how well their skills fit an internship and which required skills are missing.

3.2.6.2 Inputs. The authenticated student's skills and the internship's required and preferred skills.

3.2.6.3 Processing. The score is computed with the weighted formula in Section 4.4: required skills count three times as much as preferred skills, and an internship with no skills scores 100.

3.2.6.4 Outputs. Score (0–100), matched required skills, missing required skills, matched preferred skills and the totals of required and preferred skills; shown as a gauge and skill chips.

3.2.6.5 Error Handling. No student profile or an unknown internship returns 404 with a message; the detail page then hides the gauge.

3.2.7 FR-7 Application Submission and Tracking

3.2.7.1 Introduction. The system shall let a student apply to a published internship and follow the application until a decision is made.

3.2.7.2 Inputs. Internship identifier and cover letter; application identifier for withdrawal.

3.2.7.3 Processing. The server checks that the student has a profile and that the internship is published, then creates the application with status Applied. A database constraint on (student, internship) prevents a second application. A student may withdraw only while the status is still Applied.

3.2.7.4 Outputs. The application with internship summary, status and dates; the student's application list.

3.2.7.5 Error Handling. "Please complete your student profile before applying", "This internship is not accepting applications" and "You have already applied for this internship" are returned as 400 errors and shown as toasts. Withdrawing after review has started returns 404.

3.2.8 FR-8 Applicant Review and Status Management

3.2.8.1 Introduction. The system shall let a recruiter review applicants for their company's internships and record decisions.

3.2.8.2 Inputs. Optional internship filter; application identifier, new status (Applied, Under Review, Shortlisted, Selected, Rejected) and recruiter notes.

3.2.8.3 Processing. The applicant list is restricted to internships of companies where the recruiter has an active membership. The recruiter opens an applicant to see the student profile and cover letter, chooses a status and adds notes, which are visible only to recruiters.

3.2.8.4 Outputs. Applicant table with status tabs and counts; the updated application; the new status appears on the student's dashboard.

3.2.8.5 Error Handling. Applications of other companies are not found (404). An invalid status value returns 400. A failed update shows the toast "Failed to update status."

3.2.9 FR-9 Role-Based Dashboards and Analytics

3.2.9.1 Introduction. The system shall show each role the KPIs it needs.

3.2.9.2 Inputs. The authenticated user's role and identity.

3.2.9.3 Processing. Student: total applications, counts by status, skill count, profile completion and five most recent applications. Recruiter: total and published internships, total applicants, shortlisted, selected, rejected and five most recent applicants. University: total students, published internships, applications, placements, verified companies, placement rate (selected ÷ applications × 100, one decimal) and top ten student skills. Administrator: totals of users, students, companies, internships, applications, selections and skills. All values are computed with database aggregation (`Count`).

3.2.9.4 Outputs. JSON KPI objects rendered as stat tiles, segmented bars and Recharts charts.

3.2.9.5 Error Handling. A user without the required role receives 403 "You do not have the required role to access this resource."; superusers may open every dashboard. A student without a profile receives 404.

3.3 Non-Functional Requirements

3.3.1 Performance

- 95% of API requests shall complete in under 500 ms on the reference server with up to 100 concurrent users.
- List endpoints shall return at most 20 records per page; related objects shall be loaded with `select_related` to avoid N+1 queries.
- Dashboard KPIs shall be computed in the database with aggregation, not in Python loops.

3.3.2 Reliability

- Duplicate applications, duplicate student skills, duplicate internship skills and duplicate company memberships shall be impossible, enforced by database unique constraints.
- A failed API call shall never leave the interface blank: the user sees an error toast, and dashboards fall back to labelled sample data when enabled.

3.3.3 Availability

The production system shall be available 99% of the time per month, excluding planned maintenance announced 24 hours in advance. Public listings shall remain viewable without login.

3.3.4 Security

- Passwords shall be stored only as salted hashes using Django's password hasher.
- Every non-public endpoint shall require a valid JWT; access tokens expire after 1 hour and refresh tokens after 7 days, and refresh tokens rotate on use.
- Users shall only read or change their own profile, skills and applications, and recruiters only applications for their own companies.
- In production `DEBUG` shall be off, `SECRET_KEY` shall come from the environment,

`ALLOWED_HOSTS` and CORS origins shall list only real domains, and HTTPS shall be enforced; the OWASP Top 10 [11] is the checklist.

3.3.5 Maintainability

The backend shall keep the separation into five Django applications with models, serializers, views and URLs in separate files. Matching logic shall stay a pure function in `internships/matching.py` so it can be unit-tested and replaced. All frontend API calls shall go through the single client in `src/lib/api.ts`.

3.3.6 Portability

Configuration shall be read from environment variables so the same code runs on Windows, Linux or macOS. Switching from SQLite to PostgreSQL shall require only changing `DB_ENGINE` and the connection variables. The frontend shall work in the current versions of Chrome, Edge, Firefox and Safari and on screens from 360 px wide.

3.4 Design Constraints

- The custom user model `accounts.User` (extending `AbstractUser` with role, phone and photo) must be used throughout; it cannot be swapped after the first migration.
- The API must stay stateless (JWT), so it can be scaled horizontally behind a load balancer.
- Matching must remain rule-based and explainable for the MVP; any future ML model must still report matched and missing skills.
- The UI must use the shadcn/ui component system and Tailwind CSS design tokens for a consistent look in light and dark themes.

3.5 Other Requirements

Times are stored in UTC and displayed in Indian Standard Time (`TIME_ZONE = Asia/Kolkata`). The interface language is English (`en-us`). Uploaded files are stored in separate folders for profile photos, university logos, company logos and résumés. The database schema is managed only through Django migrations.

4. ANALYSIS MODELS

This section presents the models used to derive the requirements in Section 3. Each model is traced to the functional requirements it supports.

4.1 Data Flow Diagrams (DFD)

The Level 0 context diagram (Fig. 2) shows CareerBridge as a single process exchanging data with its four external entities. Students send profile data, skills, search filters and applications and receive listings, match scores and status updates; recruiters send postings and decisions; universities and the administrator receive aggregate statistics.

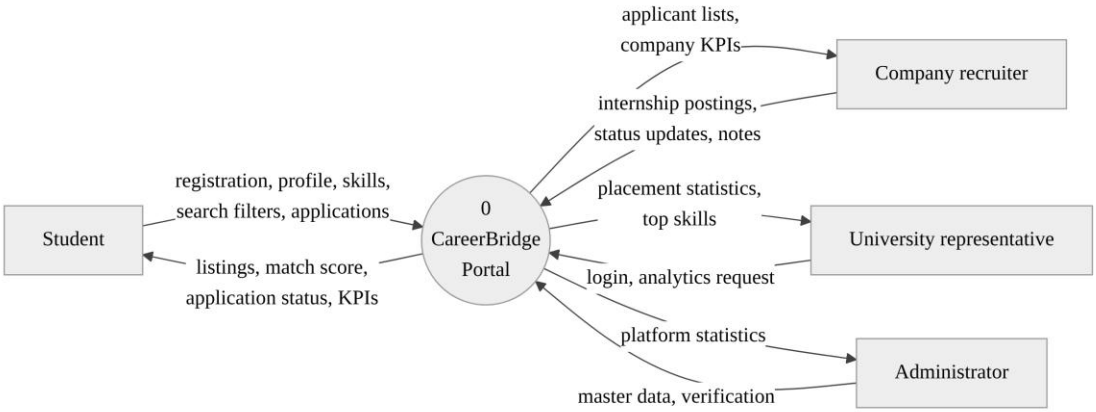


Fig. 2. Level 0 DFD (context diagram) of CareerBridge.

The Level 1 DFD (Fig. 3) decomposes the system into six processes that map directly onto the functional requirements: 1.0 Authenticate (FR-1), 2.0 Profile and skills (FR-2, FR-3), 3.0 Manage internships (FR-4), 4.0 Search and match (FR-5, FR-6), 5.0 Process applications (FR-7, FR-8) and 6.0 Generate analytics (FR-9). The six data stores D1–D6 correspond to the database tables described in Section 4.2. Process 6.0 only reads from the stores, which is why analytics never changes data.

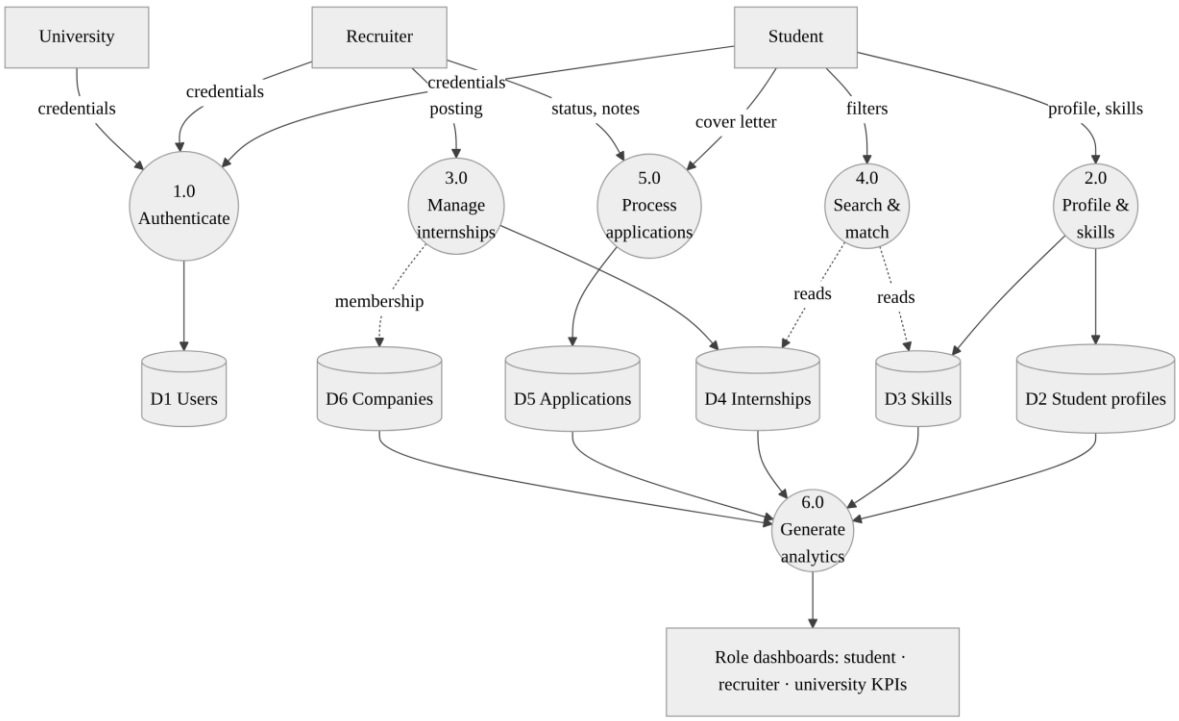


Fig. 3. Level 1 DFD showing processes, data stores and data flows.

4.2 Entity-Relationship Model

Fig. 4 shows the ten entities of the data model. A User can have at most one StudentProfile and can belong to several companies through CompanyMembership. Skills are linked to students through StudentSkill (with a proficiency level) and to internships through InternshipSkill (with an `is_required` flag), which is the data the matching algorithm compares. An Application links a StudentProfile to an Internship. Four unique constraints protect integrity: (user, company), (student, skill), (internship, skill) and (student, internship). The full field list is given in Appendix A.1.

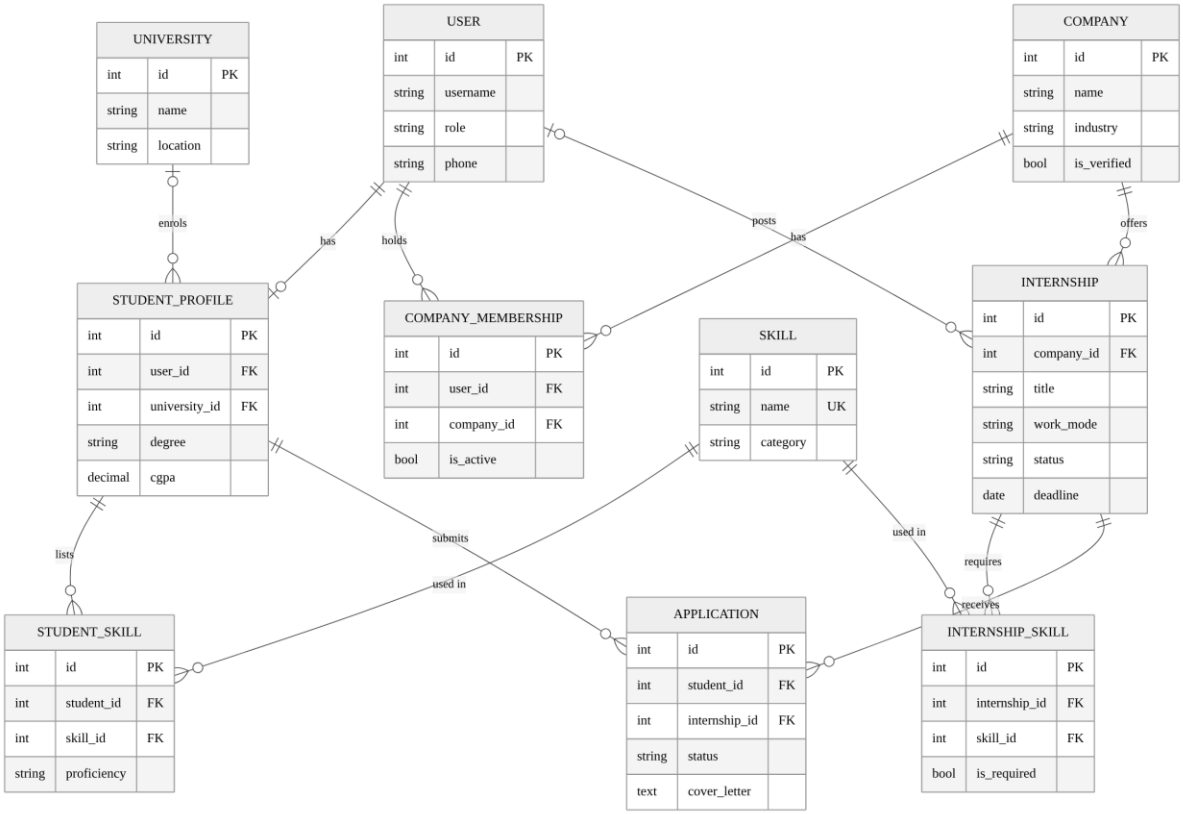


Fig. 4. Entity-relationship model of the CareerBridge database.

4.3 System Workflows

The following workflows describe how the functional requirements combine into complete journeys for each role. They are the basis for the end-to-end test scenarios.

Authentication workflow (Fig. 5). The user submits credentials; the server returns an access and a refresh token; the client stores them, loads the user's role and routes to the correct dashboard. When an access token expires, the client silently refreshes it once and repeats the request, so the user is not interrupted during the 7-day refresh window. The ShellGate component protects every `/dashboard` route and redirects visitors to `/login`, while `/internships` remains public.

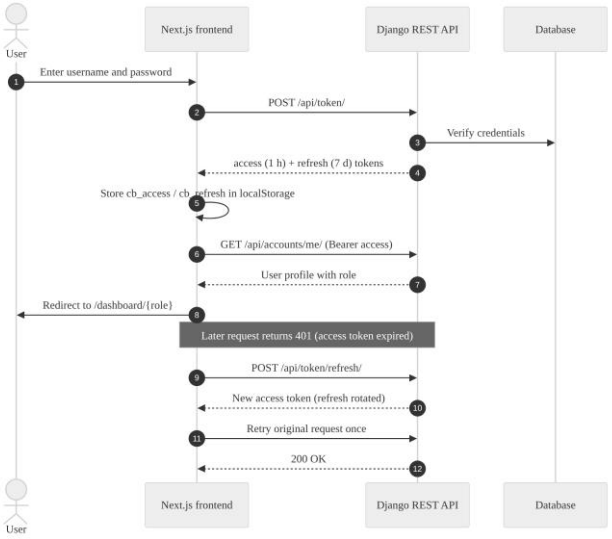


Fig. 5. Authentication and token-refresh sequence.

Student workflow (Fig. 6). The student journey has three phases. In onboarding the student registers, logs in, completes the academic profile and adds skills. In discovery the student filters listings, opens a detail page and checks the match score and missing skills, returning to the list if the fit is poor. In the apply-and-track phase the student submits a cover letter; the server rejects duplicates and unpublished listings, otherwise the application starts in Applied. The student can withdraw while it is still Applied and otherwise follows it to Shortlisted, Selected or Rejected.

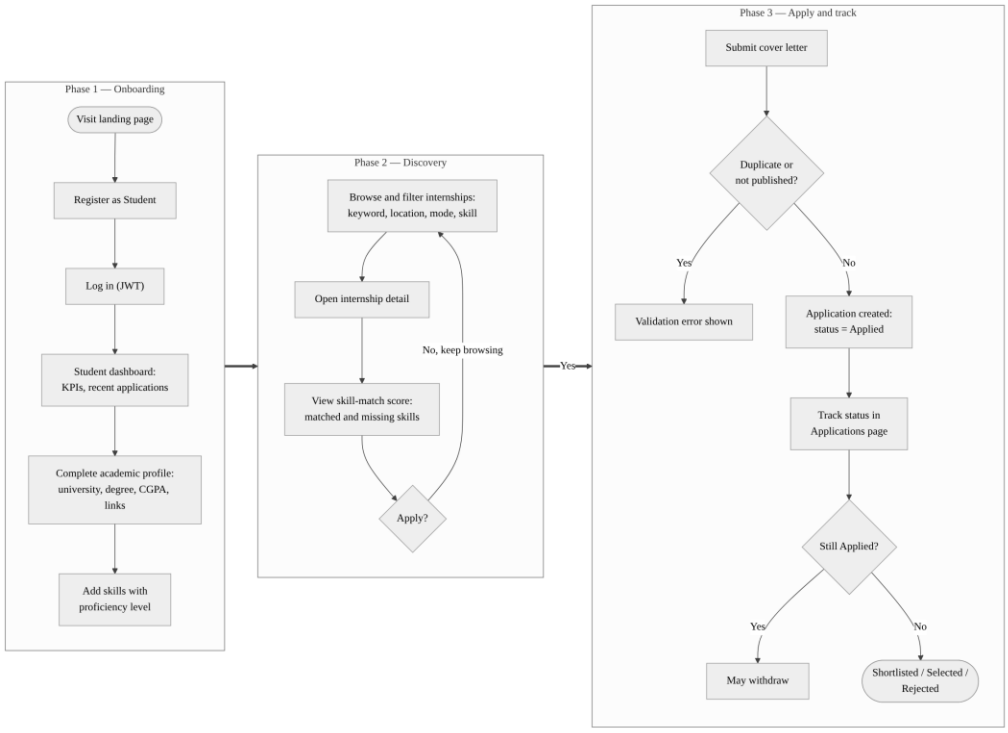


Fig. 6. End-to-end student workflow.

Recruiter workflow (Fig. 7). A recruiter must first have an active company membership, which the administrator creates. The recruiter then posts an internship either as a draft (kept private) or as published (visible at once). When students apply, the recruiter works through the Applicants page using status tabs, opens each applicant's profile and cover letter, and records a status and private notes; the change is visible to the student immediately.

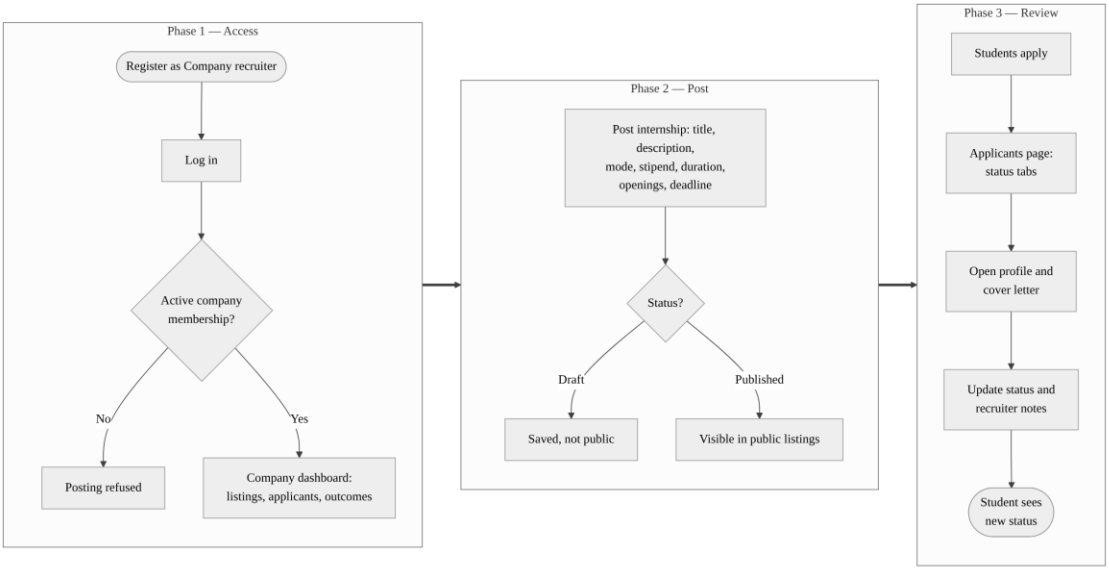


Fig. 7. End-to-end company recruiter workflow.

University workflow (Fig. 8). A university representative logs in and opens the university dashboard, which calls the university analytics endpoint. The response gives total students, published internships, applications, placements and verified companies, the placement rate and the ten most common student skills, which the placement cell uses to judge readiness and plan training.

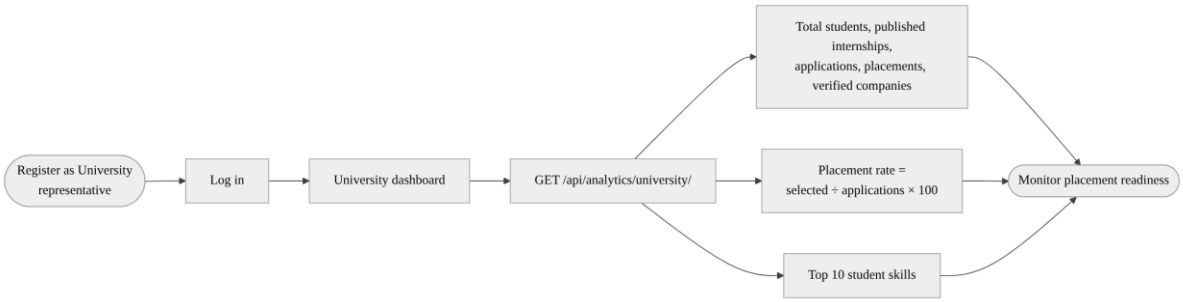


Fig. 8. University representative workflow.

Application status life cycle (Fig. 9). Every application starts in Applied. The recruiter can move it to Under Review, Shortlisted, Selected or Rejected; Selected and Rejected are final outcomes, and only Applied applications can be withdrawn by the student. The current API accepts any of the five statuses on update; enforcing the transitions in Fig. 9 on the server is listed as future work in Appendix A.2.

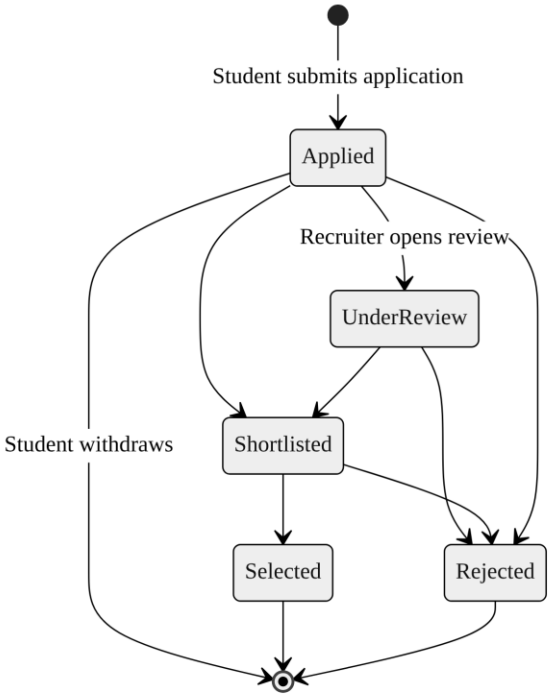


Fig. 9. Application status state diagram.

4.4 Skill-Matching Algorithm Model

The match score (FR-6) is computed by the pure function `calculate_skill_match` (Fig. 10). Let S be the set of the student's skill names, R the set of required skills and P the set of preferred skills of the internship. Then:

$$max = 3 \cdot |R| + |P|$$

$$score = round((3 \cdot |S \cap R| + |S \cap P|) / max \times 100)$$

with score = 100 when max = 0. The function also returns the matched required skills S ∩ R, the missing required skills R − S and the matched preferred skills S ∩ P, each sorted alphabetically.

Worked example. An internship requires Python, Django and SQL and prefers Docker and Git. A student has Python, SQL, Git and React. Then |S ∩ R| = 2 and |S ∩ P| = 1, so the score is round(7 / 11 × 100) = 64, and the missing required skill is Django. Running the function from the repository on these inputs returns exactly this result. A helper, `get_recommended_internships`, sorts internships by this score to produce the top-N recommendations.

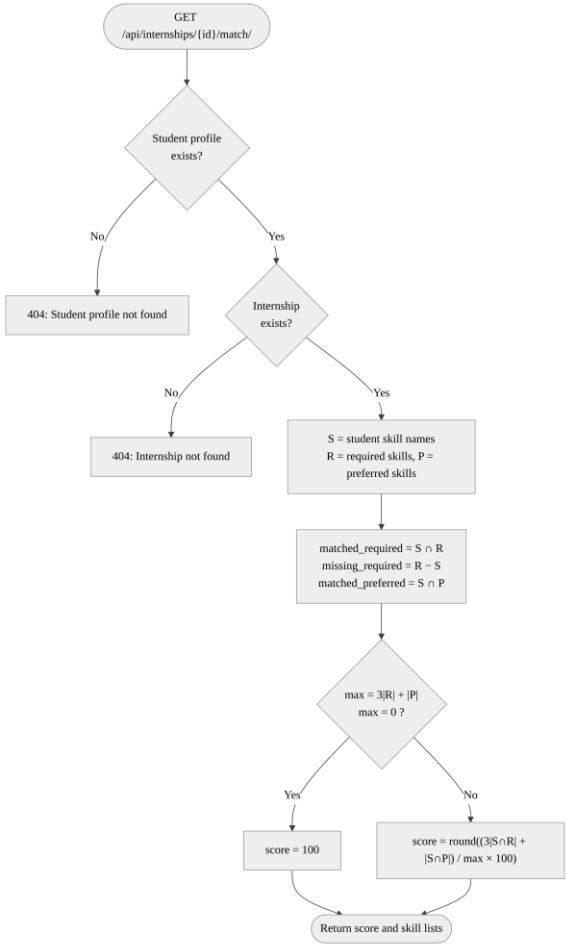


Fig. 10. Skill-matching algorithm flowchart.

5. GITHUB LINK

The complete source code of CareerBridge is maintained in the following public GitHub repository:

https://github.com/GariFFon/CareerBridge

The repository contains the Django project (`manage.py`, `requirements.txt`), the five backend applications (`accounts`, `institutions`, `internships`, `applications`, `analytics`) and the Next.js frontend in (`careerbridge-frontend/`).

Running the project locally. (1) Create and activate a Python virtual environment and run `pip install -r requirements.txt`. (2) Run `python manage.py migrate` and `python manage.py runserver` to start the API on port 8000. (3) In `careerbridge-frontend`, run `npm install` and `npm run dev` to start the frontend on port 3000. (4) Open http://localhost:3000 for the application, http://localhost:8000/api/ for the API and http://localhost:8000/admin/ for Django administration.

A. APPENDICES

The appendices give additional information. They are informative and are not part of the binding requirements in Section 3.

A.1 Appendix 1: Data Dictionary

Table VI lists every model in the database with its key fields and constraints, taken from the Django models and migrations.

TABLE VI
DATA DICTIONARY

Entity (app)	Key fields	Constraints and rules
User (accounts)	username, e-mail, first/last name, role {student, company, university, admin}, phone, profile_photo, created_at, updated_at	Custom AbstractUser; default role student; password hashed
University (institutions)	name, location, website, established_year, logo	Ordered by name
StudentProfile (institutions)	user, university, degree, branch, graduation_year, cgpa (4,2), resume_url, resume_file, linkedin_url, github_url, bio, is_available	One-to-one with User; university set to NULL if deleted; computed profile_completion %
Company (institutions)	name, website, description, industry, size {startup, small, medium, large}, location, logo, is_verified	Only verified companies are listed publicly
CompanyMembership (institutions)	user, company, is_active, joined_at	Unique (user, company)
Skill (internships)	name, category	name unique; ordered by name
StudentSkill (internships)	student, skill, proficiency {beginner, intermediate, advanced}	Unique (student, skill)
Internship (internships)	company, posted_by, title, description, location, work_mode {onsite, hybrid, remote}, stipend (10,2), duration_months (3), openings (1), deadline, status {draft, published, closed}	Ordered newest first; posted_by set to NULL if user deleted
InternshipSkill (internships)	internship, skill, is_required	Unique (internship, skill)
Application (applications)	student, internship, status {applied, review, shortlisted, selected, rejected}, cover_letter, recruiter_notes, applied_at, updated_at	Unique (student, internship); ordered newest first

A.2 Appendix 2: Code Review Observations and Future Work

While preparing this SRS the whole repository was read. Table VII records the gaps between the code at commit aded04d and the requirements above, so they can be planned rather than discovered during evaluation.

TABLE VII
CODE REVIEW OBSERVATIONS

#	Observation	Location	Recommendation
1	Default admin username and password are written in the public README and in the superuser script.	README.md, create_superuser.py	Change the password, remove it from both files and read it from an environment variable.
2	Insecure defaults: development SECRET_KEY, DEBUG on and ALLOWED_HOSTS "*".	CareerBridge/settings.py	Provide production values through a .env file; fail if SECRET_KEY is missing.
3	The registration API accepts any role, including admin, because role is a writable field.	accounts/serializers.py	Restrict role to student, company and university at registration.
4	No models are registered in Django admin, so skills, companies, universities, memberships and internship skills cannot be managed there.	*/admin.py	Register all models with list filters and search; add company verification action.
5	Internship skills cannot be set through the posting API or form, so matching has no data for new listings.	InternshipCreateSerializer, create page	Accept required and preferred skill IDs when posting.
6	Recruiter application views check only authentication, not the company role (data is still scoped by membership).	applications/views.py	Use role_required("company") as in analytics.

#	Observation	Location	Recommendation
7	Skill and application views call StudentProfile.objects.get without handling a missing profile, which returns HTTP 500.	internships/views.py, applications/views.py	Use get_or_create or return 404/400.
8	Status updates accept any value; the transitions in Fig. 9 are not enforced.	ApplicationStatusUpdateSerializer	Validate allowed transitions on the server.
9	The admin role links to /dashboard/admin, but no page exists for it.	lib/nav.ts, app/dashboard	Add the admin dashboard page using /api/analytics/admin/.
10	No automated tests: every tests.py is the empty template.	*/tests.py	Add unit tests for matching and API tests for each FR.
11	README states charts use Chart.js, but the code uses Recharts.	README.md	Correct the CV description.
12	Recommendations helper exists but has no endpoint.	internships/matching.py	Expose GET /api/internships/recommended/.

Future work. Beyond fixing the items above, planned extensions are: interview scheduling and e-mail notifications on status change; résumé parsing to suggest skills automatically; proficiency-aware and learned matching that still explains its result; university-specific analytics limited to each university's own students; company self-registration with document-based verification; and deployment with Docker, PostgreSQL, Gunicorn and WhiteNoise behind HTTPS.

REFERENCES

[1] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, IEEE, 1998.

[2] Django Software Foundation, "Django documentation." [Online]. Available: https://docs.djangoproject.com/

[3] Encode OSS, "Django REST framework." [Online]. Available: https://www.django-rest-framework.org/

[4] J. Davis et al., "Simple JWT documentation." [Online]. Available: https://django-rest-framework-simplejwt.readthedocs.io/

[5] Vercel, "Next.js documentation." [Online]. Available: https://nextjs.org/docs

[6] Meta Open Source, "React documentation." [Online]. Available: https://react.dev/

[7] shadcn, "shadcn/ui." [Online]. Available: https://ui.shadcn.com/

[8] Recharts Group, "Recharts." [Online]. Available: https://recharts.org/

[9] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[10] R. Fielding, M. Nottingham and J. Reschke, "HTTP Semantics," RFC 9110, IETF, Jun. 2022.

[11] OWASP Foundation, "OWASP Top 10 – 2021." [Online]. Available: https://owasp.org/Top10/

[12] CareerBridge project repository. [Online]. Available: https://github.com/GariFFon/CareerBridge